CCP

CS-304 Database Systems

Sports League System

Submitted By:

Yasir Hameed (24-CyS-002)

Submitted to:

Ms. Hafza Eman

Department of Computer Science

HITEC University, Taxila

Table of Contents

Part 1: Database Design

1.1 Project Overview

This project is a web-based Sports League Management System that handles the core operations of a sports league — teams, players, coaches, venues, match scheduling, standings, and seasonal data. The backend runs on PHP and MySQL, with XAMPP as the local development server.

The database is built around seven tables. Each table covers one specific concern, and they connect to each other through foreign keys. The goal was to keep the design clean — no redundant columns, no data scattered across multiple places.

1.2 Database Tables

Here is a breakdown of all seven tables along with their columns, data types, and key constraints.

Table	Column	Data Type	Constraint
coaches	CoachID	INT	PRIMARY KEY, AUTO_INCREMENT
	CoachName	VARCHAR(100)	NOT NULL
	ExperienceYears	INT	DEFAULT NULL
teams	TeamID	INT	PRIMARY KEY, AUTO_INCREMENT
	TeamName	VARCHAR(100)	NOT NULL
	City	VARCHAR(50)	DEFAULT NULL
	CoachID	INT	FK → coaches
players	PlayerID	INT	PRIMARY KEY, AUTO_INCREMENT
	PlayerName	VARCHAR(100)	NOT NULL
	Age	INT	DEFAULT NULL
	Position	VARCHAR(50)	DEFAULT NULL
	TeamID	INT	FK → teams
venues	VenueID	INT	PRIMARY KEY, AUTO_INCREMENT
	VenueName	VARCHAR(100)	DEFAULT NULL

Table	Column	Data Type	Constraint
	City	VARCHAR(50)	DEFAULT NULL
	Capacity	INT	DEFAULT NULL
seasons	SeasonID	INT	PRIMARY KEY, AUTO_INCREMENT
	SeasonYear	VARCHAR(20)	DEFAULT NULL
	StartDate	DATE	DEFAULT NULL
	EndDate	DATE	DEFAULT NULL
matches	MatchID	INT	PRIMARY KEY, AUTO_INCREMENT
	SeasonID	INT	FK → seasons
	VenueID	INT	FK → venues
	HomeTeamID	INT	FK → teams
	AwayTeamID	INT	FK → teams
	MatchDate	DATE	DEFAULT NULL
	HomeScore	INT	DEFAULT NULL
	AwayScore	INT	DEFAULT NULL
standings	StandingID	INT	PRIMARY KEY, AUTO_INCREMENT
	TeamID	INT	FK → teams
	SeasonID	INT	FK → seasons
	MatchesPlayed	INT	DEFAULT 0
	Wins	INT	DEFAULT 0
	Losses	INT	DEFAULT 0
	Points	INT	DEFAULT 0

1.3 Relationships Between Tables

The seven tables are connected through the following relationships:

- coaches → teams (1:N): One coach can manage multiple teams. teams.CoachID references coaches.CoachID.
- teams → players (1:N): One team has many players. players.TeamID references teams.TeamID.

- teams → matches as Home Team (1:N): A team can appear as the home side in multiple matches. matches.HomeTeamID references teams.TeamID.
- teams → matches as Away Team (1:N): A team can also appear as the away side. matches.AwayTeamID references teams.TeamID.
- venues → matches (1:N): One venue hosts many matches. matches.VenueID references venues.VenueID.
- seasons → matches (1:N): One season contains many matches. matches.SeasonID references seasons.SeasonID.
- seasons → standings (1:N): Standings are recorded per season. standings.SeasonID references seasons.SeasonID.
- teams → standings (1:N): Each team has a standings entry. standings.TeamID references teams.TeamID.

1.4 ER Diagram

The diagram below uses Chen notation. Rectangles represent entities, ellipses represent attributes, and diamonds represent relationships between entities. The underlined attributes are primary keys; bold italic attributes are foreign keys.

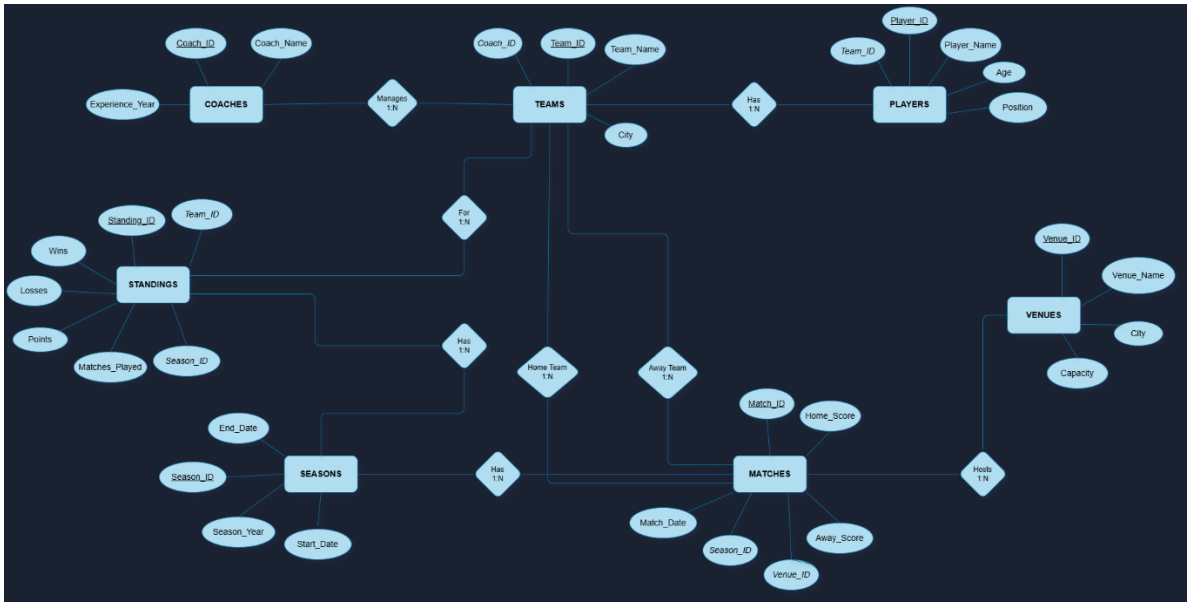

Figure 1: Entity-Relationship Diagram of the Sports League System

Part 2: Normalization (UNF → 1NF → 2NF → 3NF)

2.1 What is Normalization?

Normalization is the process of organizing a database to reduce data redundancy and avoid update anomalies. It works by progressively splitting a large, messy table into smaller, cleaner tables — each with a clear purpose.

We start with an unnormalized table (one that has everything thrown together), then work through three normal forms until the design is clean and well-structured.

2.2 Unnormalized Form (UNF)

Before any normalization, imagine storing everything about a match in a single row. That includes team details, player names, venue information, season data, and standings — all in one table. Here is what a sample record looks like:

MatchID	MatchDate	HomeTeam	HomeCity	CoachName	PlayerName	VenueName	SeasonYear	Wins	Points
1	2025-03-01	Tigers	Lahore	Ali Raza	Ali Hassan	National Stadium	2025	2	6
1	2025-03-01	Tigers	Lahore	Ali Raza	Usman Tariq	National Stadium	2025	2	6
2	2025-04-10	Eagles	Sialkot	Ahmed Khan	Bilal Ahmed	Gaddafi Stadium	2025	1	3

The problems here are obvious once you look closely. The match date, team name, city, coach name, and venue are all repeated for every player. If the city of Lahore changed, we would have to update dozens of rows. Delete a player, and we accidentally lose match information. This is exactly the kind of mess normalization fixes.

2.3 First Normal Form (1NF)

Rule: Every column must hold a single atomic value. No repeating groups.

In the UNF table, player data is repeated across rows tied to the same match — that is a repeating group. To reach 1NF, each row must be uniquely identifiable and every cell must hold one value only.

What we did: Separated players into their own rows and ensured each combination of MatchID and PlayerID forms a unique row. No multi-valued cells remain.

MatchID	MatchDate	TeamName	City	CoachName	PlayerName	Position	VenueName	SeasonYear
1	2025-03-01	Tigers	Lahore	Ali Raza	Ali Hassan	Forward	National Stadium	2025
1	2025-03-01	Tigers	Lahore	Ali Raza	Usman Tariq	Goalkeeper	National Stadium	2025
2	2025-04-10	Eagles	Sialkot	Ahmed Khan	Bilal Ahmed	Midfielder	Gaddafi Stadium	2025

The table is now in 1NF. Each cell holds a single value and each row can be uniquely identified. But there is still a problem — many columns do not depend on the full key. They only depend on part of it.

2.4 Second Normal Form (2NF)

Rule: Must be in 1NF. Every non-key attribute must depend on the whole primary key, not just part of it.

The composite key in our 1NF table is (MatchID, PlayerID). Attributes like TeamName, City, CoachName, VenueName, and SeasonYear all depend only on MatchID — not on which player is in the row. This is called a partial dependency, and it is what 2NF eliminates.

What we did: Split the table so that every attribute lives in the table where it actually belongs.

New Table	Depends On	Columns Moved Here
matches	MatchID	MatchDate, HomeTeamID, AwayTeamID, VenueID, SeasonID, HomeScore, AwayScore
teams	TeamID	TeamName, City, CoachID
players	PlayerID	PlayerName, Age, Position, TeamID
venues	VenueID	VenueName, City, Capacity
seasons	SeasonID	SeasonYear, StartDate, EndDate

After this step, every non-key attribute fully depends on the primary key of its table. No partial dependencies remain.

2.5 Third Normal Form (3NF)

Rule: Must be in 2NF. No transitive dependencies — non-key columns must not depend on other non-key columns.

After reaching 2NF, we check whether any non-key attribute depends on another non-key attribute rather than directly on the primary key. This is called a transitive dependency.

The issue we found: If we had stored CoachName and ExperienceYears directly inside the teams table, those columns would depend on CoachID (a non-key column), not on TeamID. That is a transitive dependency — TeamID → CoachID → CoachName.

Problem	Before 3NF	After 3NF
Coach attributes in teams	TeamID → CoachID → CoachName, ExperienceYears	Moved to a separate coaches table. teams only stores CoachID as a foreign key.
Result	teams was responsible for data it did not own	Each table only stores what it owns. coaches is now fully independent.

After applying 3NF, no column in any table depends on a non-key column. Every attribute answers directly to its table's primary key and nothing else.

2.6 Final Normalized Schema

The result of the complete normalization process is the seven-table schema used in this project. Each table is in 3NF:

- coaches (CoachID PK, CoachName, ExperienceYears)
- teams (TeamID PK, TeamName, City, CoachID FK)
- players (PlayerID PK, PlayerName, Age, Position, TeamID FK)
- venues (VenueID PK, VenueName, City, Capacity)
- seasons (SeasonID PK, SeasonYear, StartDate, EndDate)
- matches (MatchID PK, MatchDate, HomeScore, AwayScore, SeasonID FK, VenueID FK, HomeTeamID FK, AwayTeamID FK)
- standings (StandingID PK, MatchesPlayed, Wins, Losses, Points, TeamID FK, SeasonID FK)

Every table has a single-column primary key. Every non-key attribute depends on that key and only that key. Foreign keys enforce referential integrity across all relationships.

Part 3: SQL Script

3.1 Database Creation

The following SQL was used to create all seven tables in phpMyAdmin under the database sports_league_system. Tables are created in dependency order so that foreign key references are always valid.

```sql
CREATE TABLE `coaches` (
  `CoachID` int(11) NOT NULL,
  `CoachName` varchar(100) NOT NULL,
  `ExperienceYears` int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `coaches`
--

INSERT INTO `coaches` (`CoachID`, `CoachName`, `ExperienceYears`) VALUES
(1, 'Raza Baqir', 11),
(2, 'Ahmed Khan', 7),
(4, 'Abdul Dayan', 16),
(5, 'Yousaf Aurangzaib', 21),
(6, 'Sultan Ayyub', 33);
```

Figure 1: SQL CREATE TABLE script for the Sports League System database (excerpt)

The complete SQL script covers all seven tables with their respective primary keys, foreign key constraints, and default values.

Part 4: Portal Development

4.1 Technology Stack

The portal was built entirely with the following technologies:

- PHP — handles all server-side logic, form processing, and database queries
- MySQL (MariaDB via XAMPP) — stores all data across the seven tables
- HTML and CSS — provides the structure and styling of every page
- XAMPP — local server environment running Apache and MySQL
- phpMyAdmin — used to create tables, run queries, and verify data

4.2 User Interface

The portal uses a dark glassmorphism theme throughout. Every page shares the same navigation bar, giving the system a consistent feel. The background is a fixed sports image with a dark gradient overlay, and interactive elements glow with a cyan highlight on hover.

The dashboard serves as the home page. It shows live record counts for all seven tables, and each card links directly to the management page for that table. The design was intentional — someone should be able to navigate anywhere from anywhere without getting lost.

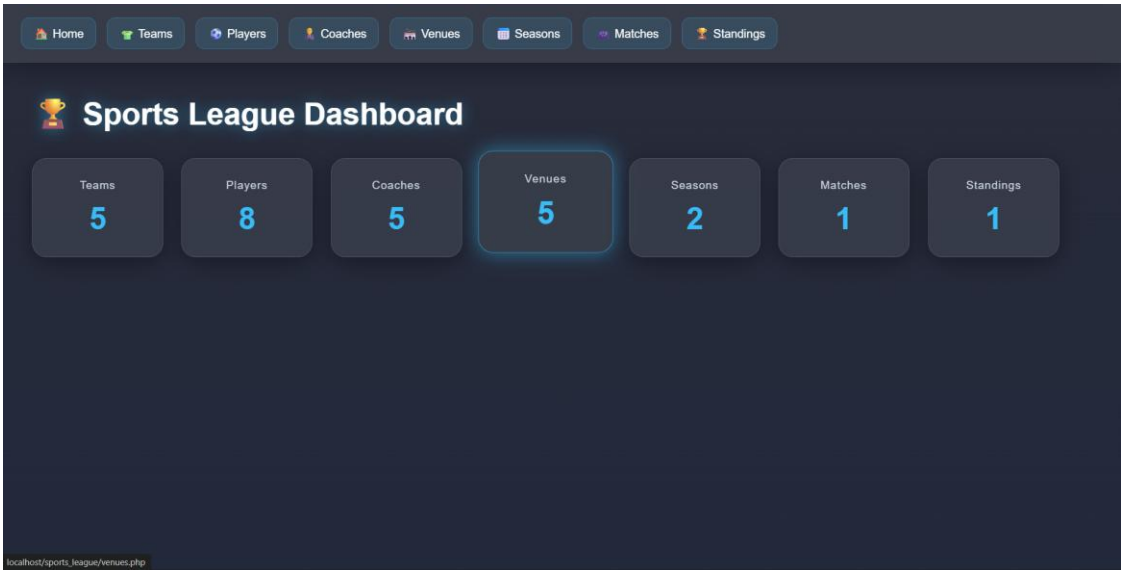

Figure 3: Sports League Dashboard with live record counts for all 7 tables

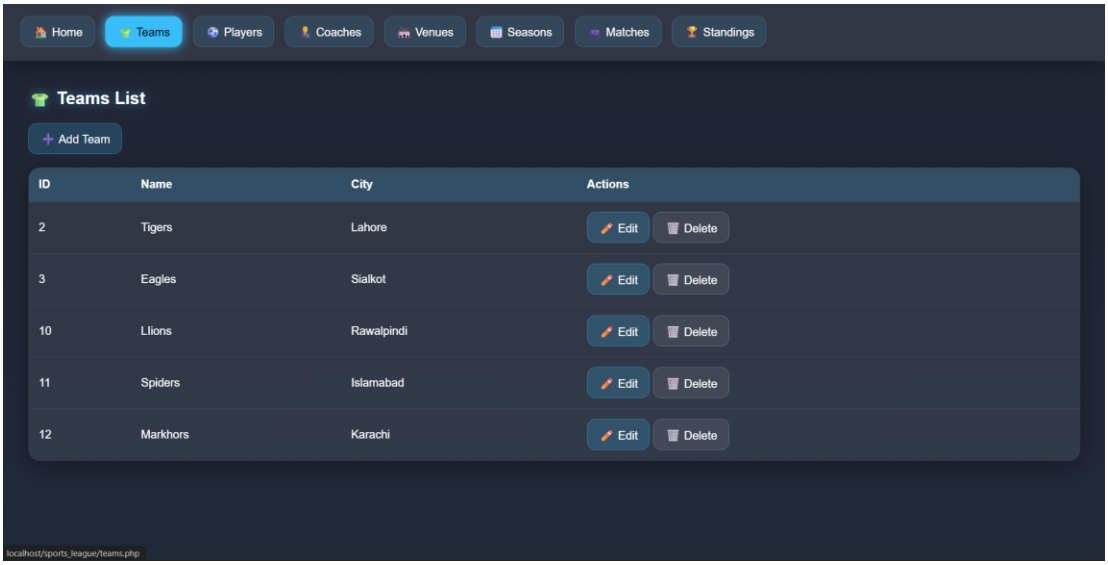

Figure 4: Teams management page with Edit and Delete actions

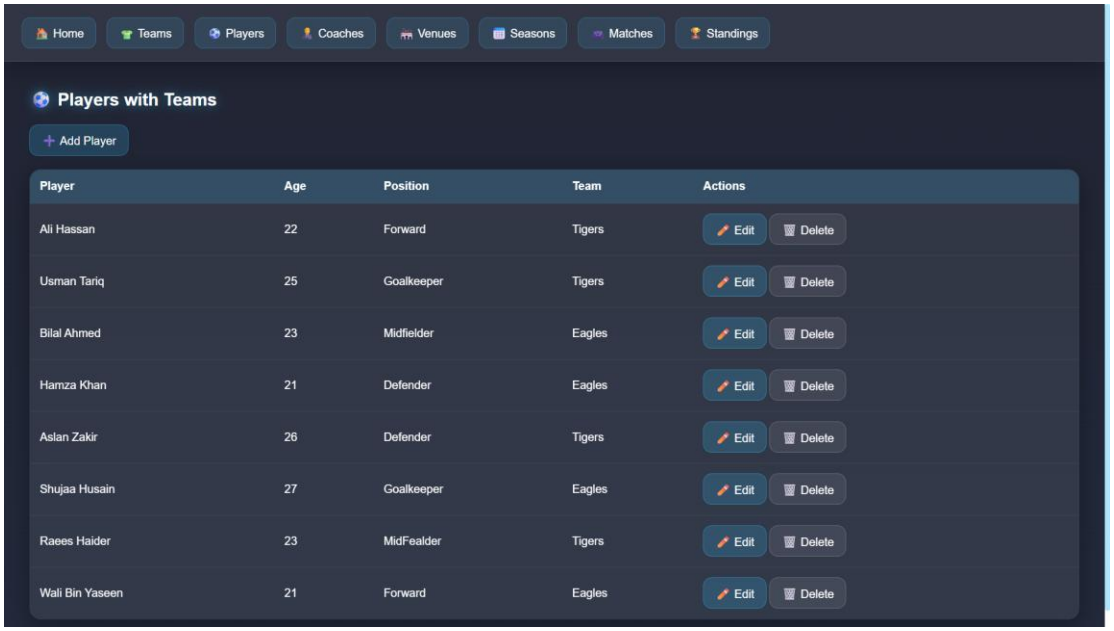

Figure 5: Players page showing JOIN result — PlayerName, Age, Position, and TeamName

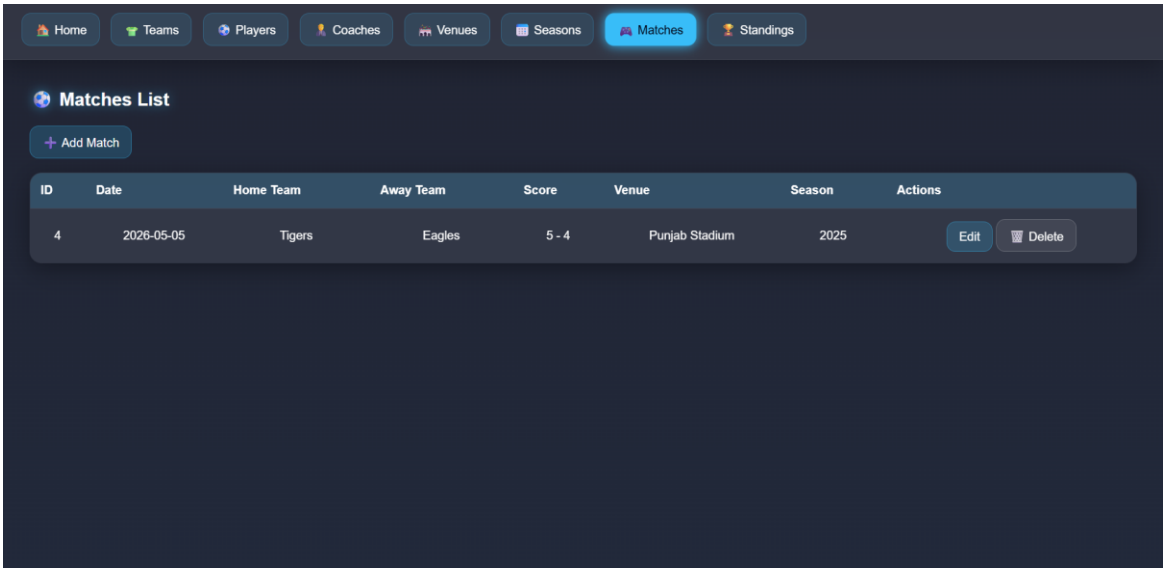

Figure 6: Matches page with multi-table JOIN showing HomeTeam, AwayTeam, Venue, and Season

4.3 Navigation

The navigation bar appears on every page and links to all seven modules. Within each module, list pages include an Add button at the top and Edit/Delete buttons on each row. Delete actions require confirmation through a browser dialog before proceeding.

Part 5: CRUD Operations

5.1 Create

New records are added through dedicated form pages for each table. Each form submits via HTTP POST to the same PHP file, which validates the input before executing an INSERT query.

Example — inserting a new player:

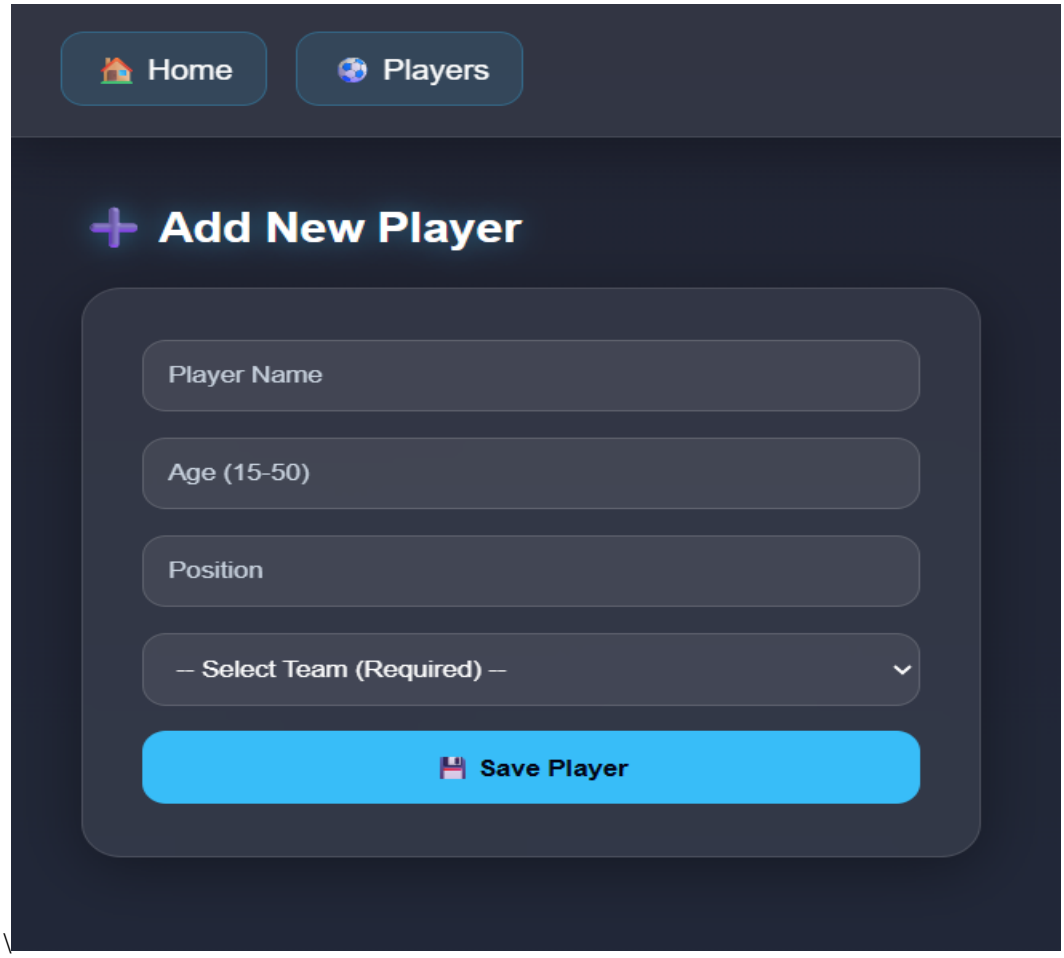

Figure 7: Add Player form with Team dropdown and validation

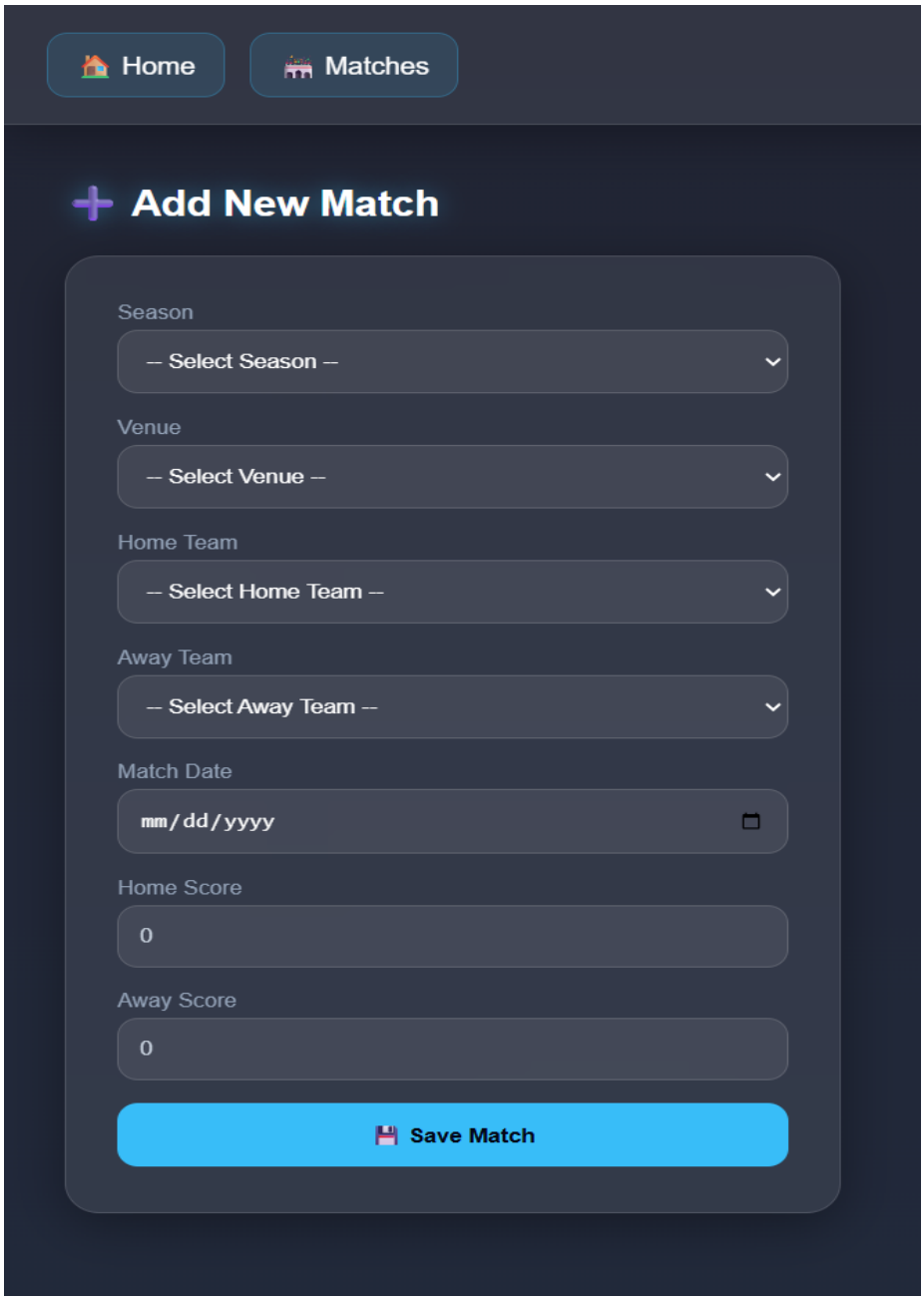

Figure 8: Add Match form with Season, Venue, Home Team, and Away Team dropdowns

5.2 Read — Using Joins

Data is read using SELECT queries. The Players page uses a LEFT JOIN between the players and teams tables, displaying each player alongside their team name in a single view.

The Matches page goes further, joining four tables at once to display meaningful information rather than raw IDs.

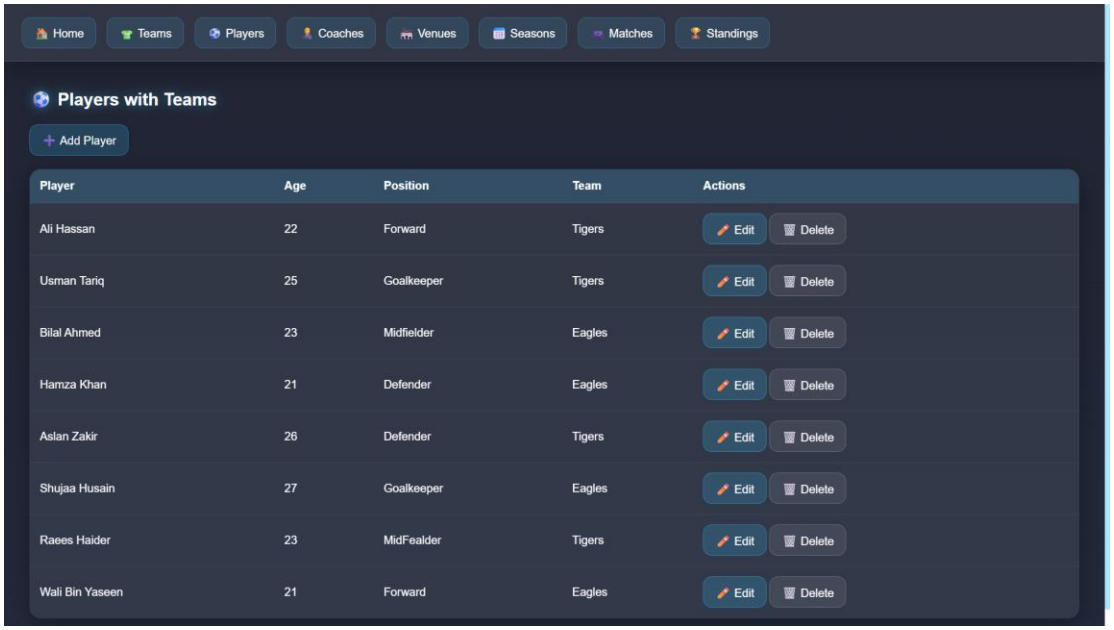

Figure 9: Players page displaying PlayerName, Age, Position, and TeamName — result of LEFT JOIN between players and teams tables

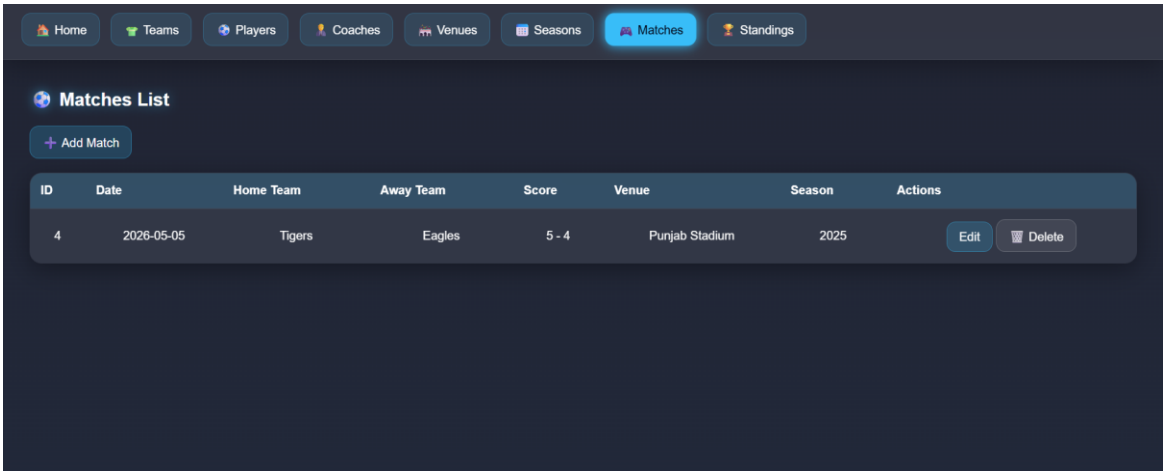

Figure 10: Matches page displaying HomeTeam, AwayTeam, Venue, and Season — result of LEFT JOIN across four tables

5.3 Update

Each record can be edited through an Edit form that comes pre-filled with the current data. On submission, an UPDATE query modifies only that record.

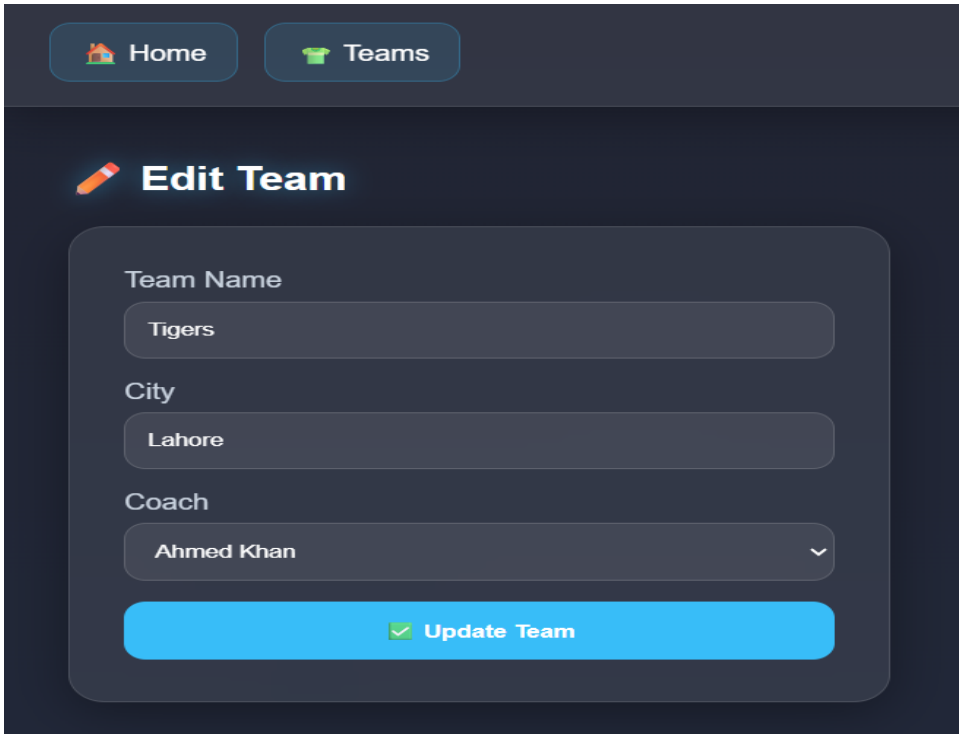

Figure 11: Edit Team form pre-filled with existing data and Coach dropdown

5.4 Delete

Delete links appear on every list page. Before anything is removed, the browser asks the user to confirm. The delete script then handles foreign key dependencies first — nullifying or removing related rows — before deleting the target record.

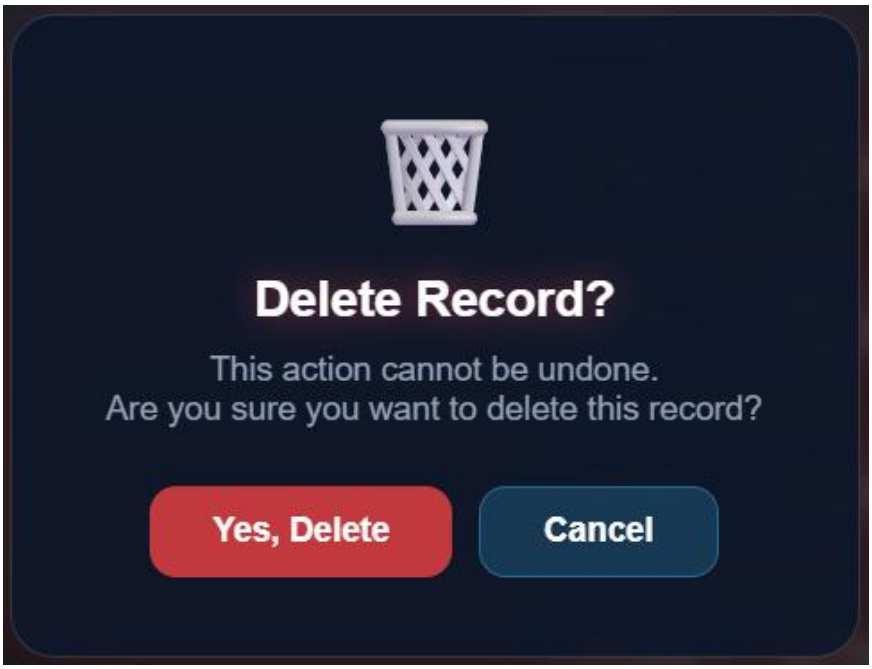

Figure 12: Delete confirmation dialogue prompting the user before a record is permanently removed

Part 6: Error Handling and Validation

6.1 Validation Strategy

Validation runs on two levels. HTML attributes handle the most basic checks in the browser so the form never submits with obviously wrong data. PHP then re-validates on the server side, which matters because anyone can bypass HTML restrictions if they know what they are doing.

When a validation error occurs, the form re-renders with a red error message at the top. The user's previously typed values are preserved so they do not have to start over.

6.2 Validation Rules by Form

Form	Validation Applied
Add / Edit Team	All fields required. Team Name and City must contain letters only. Coach selection is mandatory. Duplicate team name check against the database.
Add / Edit Player	All fields required. Player Name and Position must contain letters only. Age must be between 15 and 50.
Add / Edit Coach	All fields required. Coach Name must contain letters only. Experience must be between 0 and 50 years. Duplicate name check against the database.
Add / Edit Venue	All fields required. Venue Name and City must contain letters only. Capacity must be a number between 100 and 200,000.
Add / Edit Season	All fields required. Season Year must be a valid 4-digit number between 2000 and 2100. End Date must be later than Start Date.
Add / Edit Match	Season, Venue, Home Team, Away Team, and Date are all required. Home Team and Away Team cannot be the same. Scores must be 0 or greater (no upper limit — cricket scores can exceed 500).
Add / Edit Standing	Team and Season selection required. Wins + Losses cannot exceed Matches Played. Points must be 0 or greater.
Delete (all tables)	A confirmation dialog appears before any record is deleted, preventing accidental data loss.

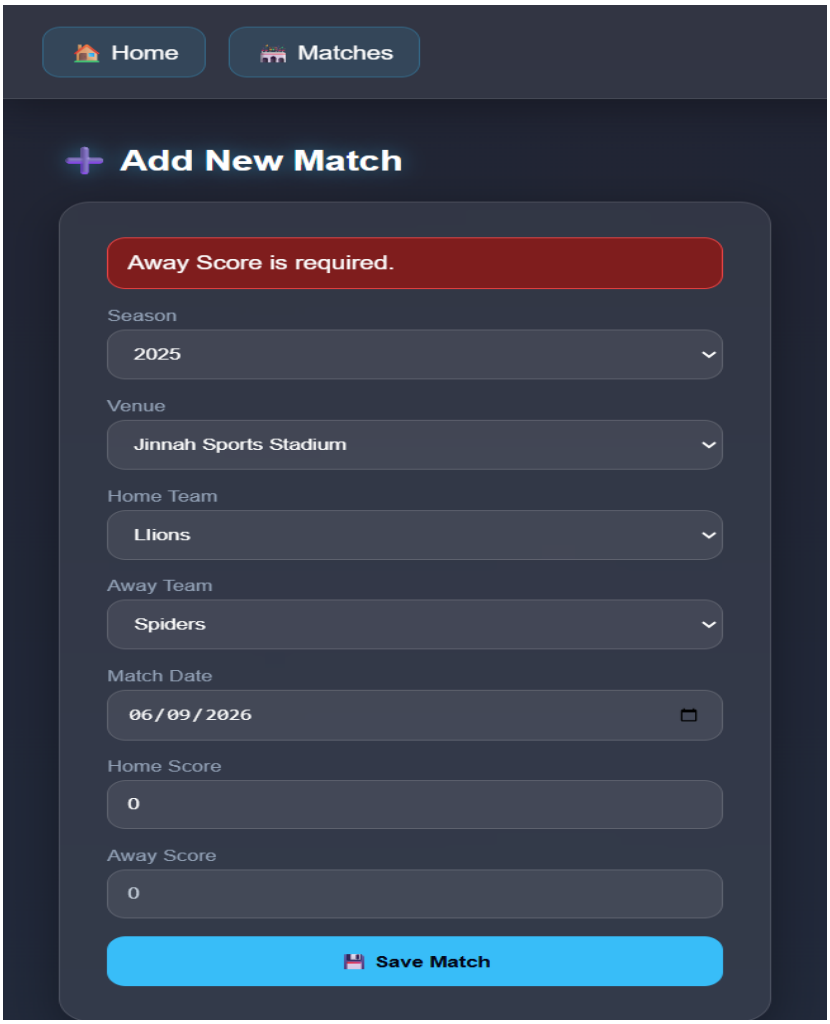

Figure 13: Red error message shown when validation fails, with form data preserved

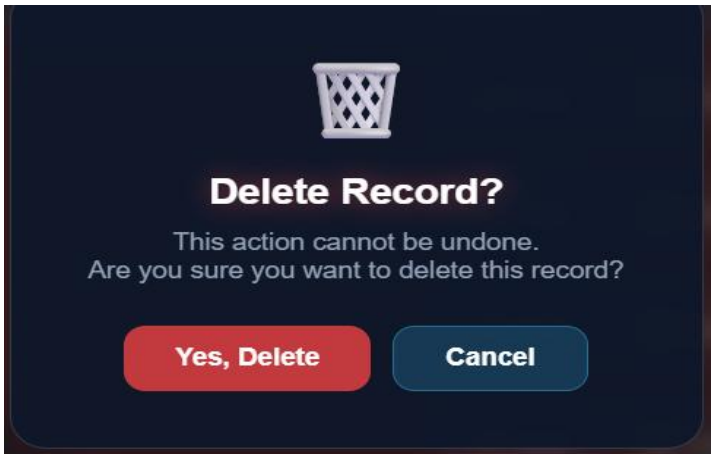

Figure 14: Browser confirmation dialog before a team is deleted

Part 7: Additional Screenshots

7.1 All Table Management Pages

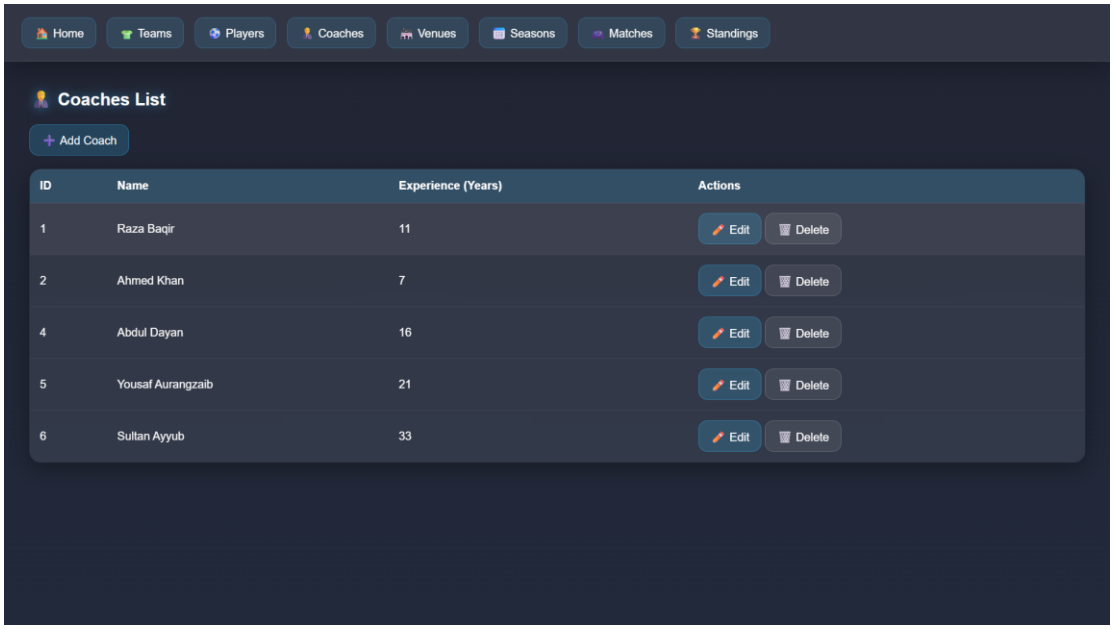

Figure 15: Coaches management page

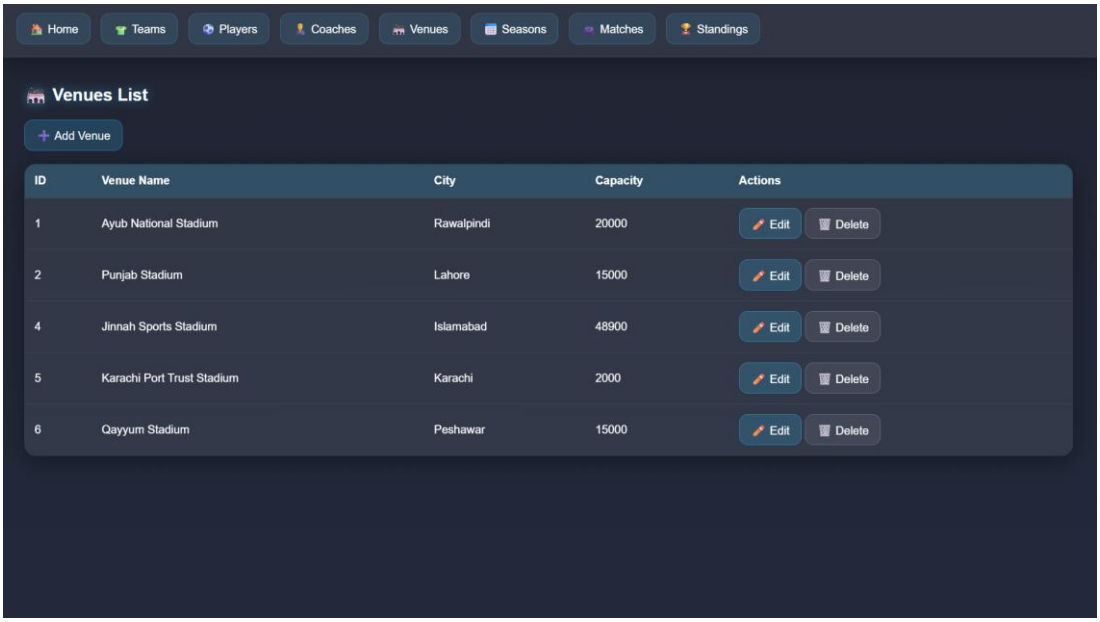

Figure 16: Venues management page

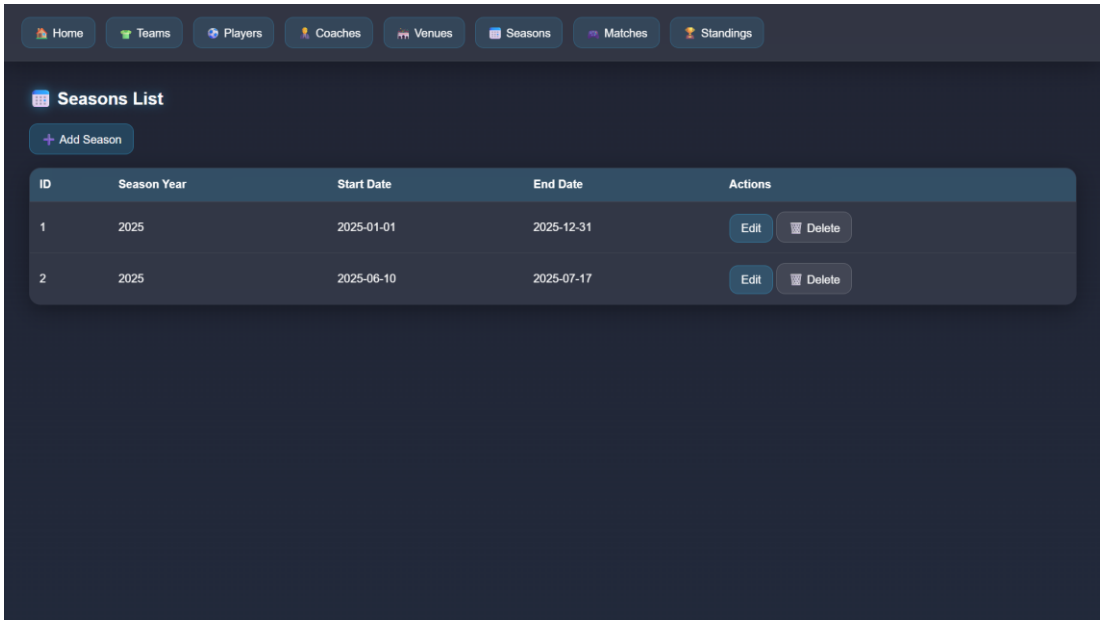

Figure 17: Seasons management page

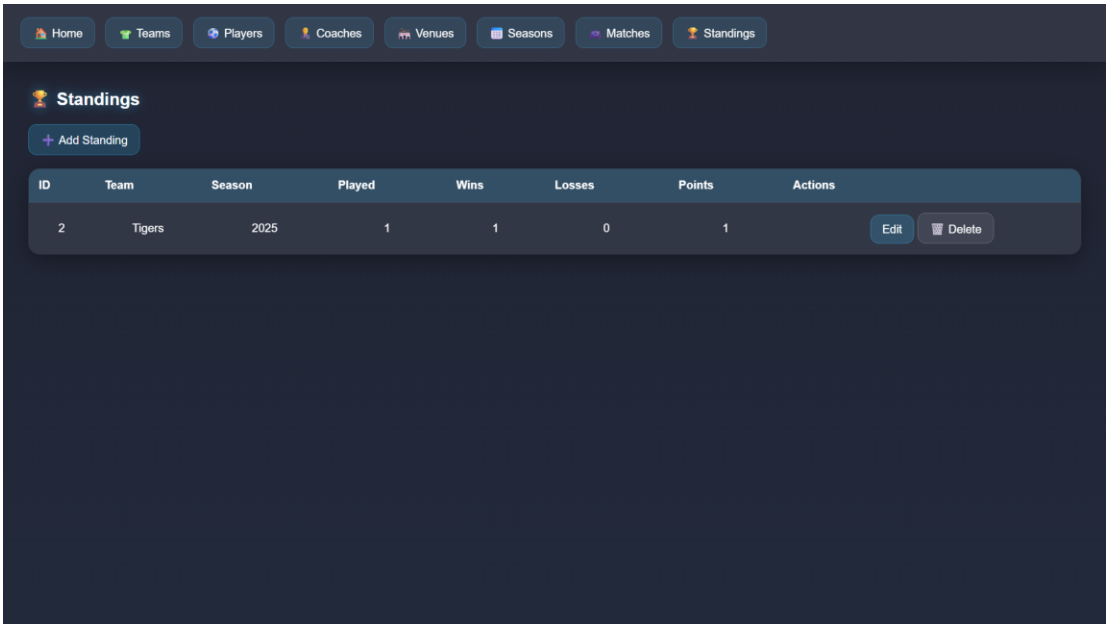

Figure 18: Standings page ordered by points descending

7.2 phpMyAdmin Views

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	MatchID	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	SeasonID	int(11)			Yes	NULL			Change Drop More
3	VenueID	int(11)			Yes	NULL			Change Drop More
4	HomeTeamID	int(11)			Yes	NULL			Change Drop More
5	AwayTeamID	int(11)			Yes	NULL			Change Drop More
6	MatchDate	date			Yes	NULL			Change Drop More
7	HomeScore	int(11)			Yes	NULL			Change Drop More
8	AwayScore	int(11)			Yes	NULL			Change Drop More

Figure 19: Table structure showing column names, data types, and key constraints

Table structure | Relation view

Foreign key constraints

Actions	Constraint properties	Column	Foreign key constraint (INNODB) Database	Table	Column
Drop	matches_ibfk_1 ON DELETE RESTRICT ON UPDATE RESTRICT	SeasonID + Add column	sports_league_syst	seasons	SeasonID
Drop	matches_ibfk_2 ON DELETE RESTRICT ON UPDATE RESTRICT	VenueID + Add column	sports_league_syst	venues	VenueID
Drop	matches_ibfk_3 ON DELETE RESTRICT ON UPDATE RESTRICT	HomeTeamID + Add column	sports_league_syst	teams	TeamID
Drop	matches_ibfk_4 ON DELETE RESTRICT ON UPDATE RESTRICT	AwayTeamID + Add column	sports_league_syst	teams	TeamID
	Constraint name ON DELETE RESTRICT ON UPDATE RESTRICT	 + Add column	sports_league_syst		

+ Add constraint

Figure 20: Foreign key constraints visible in phpMyAdmin

Conclusion

This project involved designing and developing a fully functional Sports League Management System using PHP and MySQL. The database was built from scratch — starting with an unnormalized structure and progressively applying 1NF, 2NF, and 3NF to arrive at a clean, seven-table schema with well-defined relationships and foreign key constraints. A complete web portal was developed on top of this database, covering all seven entities with full CRUD functionality, input validation, and error handling across every form. JOIN queries were implemented to pull meaningful data from multiple tables simultaneously, as seen in the Players and Matches pages. The system was tested locally using XAMPP, with phpMyAdmin used to verify data integrity throughout development. Overall, this project provided hands-on experience with real-world database design principles and the practical challenges that come with building a data-driven web application.